

Constructor init function placement in TRENTOS core API

Page Content

- Option 1: Generic `_init()` with type selector and a fully comprehensive struct that would include all of the possible needed parameters for all of the implementation.
- Option 2: Generic `_init()` with type selector and abstract data type (ADT) pointer to the context needed to the specific constructor.
- Option 3: Generic `_init()` with type selector and union data type containing all of the needed parameters for each Implementation.
- Option 4: Generic `_init()` with type selector and context in `va_args`.
- Option 5: Specialized `_init()` in implementation dedicated headers.
 - Code example

Keystore multi implementation (see [SEOS-2731 - Getting issue details...](#) STATUS) is showing some limitation in the current code base in regard of constructors. The issue is that while we can abstract in an API the operations of a give component, we cannot abstract the constructor. The constructor has always in the signature implementations details. The following one here below, for example, assumes you want to use a Filesystem and this, is not something we are going to have always.

OS_Keystore.h

```
OS_Error_t
OS_Keystore_init(
    OS_Keystore_Handle_t* hKeystore, /**< [in,out] Pointer to the keystore
                                   handle. */
    OS_FileSystem_Handle_t hFs,      /**< [in] Keystore's file system handle.*/
    OS_Crypto_Handle_t hCrypto,      /**< [in] Handle to initialized the
                                   OS_Crypto instance. */
    const char* name                /**< [in] Name of the given keystore. */);
```

To deal with this situation there is a limited amount of options:

Option 1: Generic `_init()` with type selector and a fully comprehensive struct that would include all of the possible needed parameters for all of the implementation.

That would be in pseudocode, in `OS_Keystore.h`:

OS_Keystore.h

```
OS_Error_t
OS_Keystore_init(
    OS_Keystore_Handle_t hKeystore,
    enum keystore type,
    struct allOfThePossibleParameters);
```

PRO	CONS
+ Generic init-API that hides implementation completely from user + compile time type checking on construction parameters/context	- specialization dependencies in <code>OS_Keystore.h</code> (both the enum and the context) - readability (would be hard to check that we are giving the right parameters od the we are completed in giving them just looking at the <code>init()</code> call line of code) - maintainability (touching a file or a data structure on behalf of many implementations makes maintenance harder: merges, git log etc.) - scalability (in coding) - single responsibility broken at the data type level (one struct for all implementations) - ifdef anti-pattern necessary in generic constructor (in case an implementation is excluded at compile time to reduce TCB)

Option 2: Generic `_init()` with type selector and abstract data type (ADT) pointer to the context needed to the specific constructor.

That would be in pseudo-code, in `OS_Keystore.h`:

OS_Keystore.h	
<pre>OS_Error_t OS_Keystore_init(OS_Keystore_Handle_t* hKeystore, enum keystore type, void* context);</pre>	
PRO	CONS
 Generic init-API that hides implementation completely from user	 specialization dependencies in OS_Keystore.h (only the enum, but still a dependency)  no compile time type checking on construction parameters/context  ifdef anti-pattern in generic constructor (in case an implementation is excluded at compile time)  It could result in crashes when trying to access a wrong data structure (even if immediately detectable once at runtime it is still a very negative perspective)

Option 3: Generic `_init()` with type selector and union data type containing all of the needed parameters for each implementation.

That would be in pseudocode, in `OS_Keystore.h`:

OS_Keystore.h	
<pre>OS_Error_t OS_Keystore_init(OS_Keystore_Handle_t* hKeystore, enum keystore type, union configAndContext);</pre>	
PRO	CONS
 Generic init-API that hides implementation completely from user  here we cannot really state that we have strong compile time type checking as the union is a blob	 specialization dependencies in OS_Keystore.h (both the enum and the context)  ifdef anti-pattern in generic constructor (in case an implementation is excluded at compile time)  It could result in crashes when trying to access a wrong data structure (even if immediately detectable once at runtime it is still a very negative perspective)

Option 4: Generic `_init()` with type selector and context in `va_args`.

That would be in pseudocode, in `OS_Keystore.h`:

OS_Keystore.h	
<pre>OS_Error_t OS_Keystore_init(OS_Keystore_Handle_t* hKeystore, enum keystore type, ...);</pre>	
PRO	CONS

	➖ specialization dependencies in OS_Keystore.h (only the enum, but still a dependency) ➖ no compile time type checking on construction parameters/context ➖ ifdef anti-pattern in generic constructor (in case an implementation is excluded at compile time) ⚠️ no checking of correct amount of arguments possible and could result in crashes (even if immediately detectable once at runtime it is still a very negative perspective)
--	--

Option 5: Specialized _init() in implementation dedicated headers.

The init() function in this scenario is removed from **OS_Keystore.h** the user module that wants to initiate a specialized Keystore has to include its header file.

Given for example the name of the specialization "Impl1" and an integer and a char as construction parameters, that would be in pseudocode, in **OS_KeystoreImpl1.h**:

OS_KeystoreImpl1.h
<pre>#include "OS_Keystore.h" OS_Error_t OS_KeystoreImpl1_init(OS_Keystore_Handle_t* hKeystore, int specializationParamInt, char specializationParamChar);</pre>

PRO	CONS
➕ no specialization dependencies in OS_Keystore.h ➕ compile time type checking on construction parameters/context ➕ use of a common pattern to implement interfaces	❓ Is API broken because there is no generic Init function? (User has to include header and call init of used implementation.)

Code example

Supposing to have a business logic module called Crypto.c that provides cryptography functions which have to use keys stored in the keystore. The Crypto.c module will be provided with a signature

like

<pre>#include "OS_Keystore.h" OS_Error_t Crypto_init(Crypto* self, OS_Keystore_Handle_t hKeystore);</pre>
--

The initialization part of the full application can stay in a separate file from the business logic modules like Crypto.c. Let's say having an Init.c file. The the code will look like:

<pre>#include "OS_KeystoreImpl1.h" #include "Crypto.h" OS_Keystore_Handle_t* hKeystore = NULL; Crypto* crypto = NULL; OS_Error_t err = OS_KeystoreImpl1_init(hKeystore, 0, 'a'); if (err != OS_SUCCESS) return -1; err = Crypto_init(&crypto, *hKeystore);</pre>

The important part in that code example is that when the business logic part is separated (Crypto.c) and the composition is made correctly (Crypto take a pointer to an abstract object Keystore) all of the business logic can use only OS_Keystore_* functions, that is the abstract interface of OS_Keystore defined in OS_Keystore.h. There is no dependency in Crypto.c to OS_KeystoreImpl1.h ! Hence OS_Crypto.c is robust (invariant) to the changes of implementation of Keystore.